



Trailhead



[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Work with Salesforce Data

Work with Salesforce Data

Learning Objectives

After completing this unit, you'll be able to:

- Use base components to create forms to work with single records.
- Use the wire service to get data.
- Create and update data.
- Call Apex methods.

A component without any data might look good, but it feels a little empty inside. In this unit, we fill that void and compare how Aura and Lightning components read and write Salesforce data.

If you developed Apex classes for your Aura components, you can reuse the Apex in Lightning web components. However, because of the alternatives for working with data in Lightning web components, you might find that Apex is no longer necessary.



If you'd like to learn even more about working with Salesforce data, visit the [Lightning Web Components and Salesforce Data \(https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-and-salesforce-data\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-and-salesforce-data) module.

Create a Form to Work with Single Records

To create forms that let users view, edit, and create Salesforce records, Aura components and Lightning web components can both use base components in the lightning namespace.

These components provide a form layout and handle record CRUD changes without



Hi NELLURI, ask me about products, features, and pricing, or connect to a sale...





we've seen earlier for Aura components and Lightning web components.

Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

Work with Salesforce Data

Form Function	Aura component	Lightning web component
Support edit, view, and read-only modes	lightning:recordForm	lightning-record-form
Read-only form	lightning:recordViewForm	lightning-record-view-form
Editable form	lightning:recordEditForm	lightning-record-edit-form

For most use cases, lightning-record-form provides a great starting point. It combines and simplifies the functionality of lightning-record-view-form and lightning-record-edit-form.

For more advanced use cases that require custom field layouts and custom rendering of record data, use lightning-record-view-form and lightning-record-edit-form.

Here's an example in the BrokerDetails Aura component.

```

1 <lightning:recordForm
2   objectApiName="Broker__c"
3   recordId="{!v.property.Broker__c}"
4   fields="{!v.brokerFields}"
5   columns="2"/>

```



Here's equivalent HTML in the brokerCard Lightning web component.





Custom UI to Work with Single Records

Aura components that need more control over the UI than `lightning:recordForm` and `lightning:recordEditForm` can use the `<force:recordData>` tag with their own custom UI in markup.

The Property Summary Aura component uses `<force:recordData>` in its markup.

```

1 <force:recordData aura:id="service"
2     recordId="{!v.recordId}"
3     targetFields="{!v.property}"
4     fields="['Id',
5             'Thumbnail__c',
6             'Address__c',
7             'City__c',
8             'State__c',
9             'Zip__c',
10            'Price__c',
11            'Beds__c',
12            'Baths__c',
13            'Broker__r.Id',
14            'Broker__r.Name',
15            'Broker__r.Title__c',
16            'Broker__r.Mobile_Phone__c',
17            'Broker__r.Email__c',
18            'Broker__r.Picture__c']" />

```



Instead of `<force:recordData>`, Lightning web components have a few techniques that use Lightning Data Service, but differ depending on whether you are reading or writing data. Consider using the wire service and JavaScript API methods only if the lightning-record-* form components don't meet your requirements.

Use the Wire Service to Get Data

To read Salesforce data, Lightning web components use a reactive wire service, which is built on Lightning Data Service. Components use `@wire` in their JavaScript class to read data from one of the wire adapters in the `lightning/ui*Api` namespace. For the list of wire adapters that Salesforce provides, see the Resources section of this unit. You can't





Here's an example in `propertySummary.js` that uses `@wire` to retrieve a record.

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Work with Salesforce Data

```
1 import { LightningElement, api, wire } from 'lwc';
2 import { getRecord, getFieldValue } from
3 'lightning/uiRecordApi';
4 import NAME_FIELD from '@salesforce/schema/Property__c.Name';
5 import PICTURE_FIELD from
6 '@salesforce/schema/Property__c.Picture__c';
7 export default class PropertySummary extends LightningElement {
8     @api recordId;
9     propertyName;
10    pictureURL;
11    @wire(getRecord, { recordId: '$recordId', fields:
12 [NAME_FIELD, PICTURE_FIELD] })
13    wiredRecord({ error, data }) {
14        if (data) {
15            this.propertyName = getFieldValue(data, NAME_FIELD);
16            this.pictureURL = getFieldValue(data,
17 PICTURE_FIELD);
18        } else if (error) {
19            // Handle error. Details in error.message.
20        }
21    }
22 }
```

The code imports the `getRecord` wire adapter from `lightning/uiRecordApi`.

When you use the `ui*Api` wire adapters, we strongly recommend importing references to objects and fields. Importing references to objects and fields ensures that your code works because Salesforce verifies that the objects and fields exist. This line of code imports a reference to the `Name` field in the `Property` custom object.

```
1 import NAME_FIELD from '@salesforce/schema/Property__c.Name';
```

The `@api` decorator makes the `recordId` property public. The parent component that contains `propertySummary` sets the value of the `record-id` attribute in its HTML file.





Use JavaScript API Methods to Write Data

The `createRecord` JavaScript API creates a record. You can also use `updateRecord` on records for developers. While creating Lightning Data Service records for a platform developer is required.

Work with Salesforce Data



Important

Don't use @wire to create, update, or delete a record. The wire service delegates control flow to the Lightning Web Components engine. Delegating control is great for read operations, but it isn't great for create, update, and delete operations. As a developer, you want complete control over operations that change data. That's why you perform create, update, and delete operations with a JavaScript API instead of with the wire service.

Here's an `LdsCreateRecord` Lightning web component that uses `createRecord` to create an account.

```

1  import { LightningElement } from 'lwc';
2  import { ShowToastEvent } from
3  'lightning/platformShowToastEvent';
4  import { createRecord } from 'lightning/uiRecordApi';
5  import ACCOUNT_OBJECT from '@salesforce/schema/Account';
6  import NAME_FIELD from '@salesforce/schema/Account.Name';
7  export default class LdsCreateRecord extends LightningElement {
8      accountId;
9      name;
10     onNameChange(event) {
11         this.name = event.target.value;
12     }
13     createAccount() {
14         const recordInput = {
15             apiName: ACCOUNT_OBJECT.objectApiName,
16             fields: {
17                 [NAME_FIELD.fieldApiName]: this.name

```





createRecord returns a Promise object that resolves on success with the created record.

For more information on the JavaScript API, see the Resources at the end of this unit.

Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
The HTML file of `LdsCreateRecord` includes a button that invokes `createAccount()` in the JavaScript file. The account name is set in a lightning-input component.

Work with Salesforce Data

```
1 <template>
2   <lightning-card title="LdsCreateRecord" icon-
3   name="standard:record">
4     <div class="slds-m-around_medium">
5       <lightning-input label="Id" disabled value=
6       {accountId}></lightning-input>
7       <lightning-input label="Name" onchange=
8       {onNameChange} class="slds-m-bottom_x-small"></lightning-input>
9       <lightning-button label="Create Account"
       variant="brand" onclick={createAccount}></lightning-button>
      </div>
    </lightning-card>
  </template>
```



Use Apex for Custom Data Access

We've looked at a few techniques that use the Lightning Data Service to work with data. These techniques usually mean less code because you don't need any Apex. However, if you already have Apex code that you developed for Aura components, you can reuse it in Lightning web components. Alternatively, If you need a SOQL query for custom data access, you have to use an Apex method.

To access an Apex controller from an Aura component, make a call from the JavaScript controller or helper. Here's the helper of the PropertyCarousel Aura component calling a `getPictures()` Apex method.

```
1 ({
2   loadPictures : function(component) {
3     var propertyId = component.get("v.recordId");
4     component.set("v.files", []);
```





```

14         var state = response.getState();
15         if (state === "SUCCESS") {
16             var developer = $content.getRelatedRecord("Lightning Web Components for Aura Developers", {
17                 trailmix_creator_id: "rebelo78", trailmix_slug: "salesforce-platform-developer"
18             });
19         }
20         else if (state === "INCOMPLETE") {
21             // handle incomplete state

```

Show More

Any Aura component that accesses Salesforce data has similar boilerplate. The syntax to access Apex methods is different for Lightning web components.

Lightning web components can import methods from Apex classes. The imported methods are functions that the component can call declaratively with @wire or directly in code.

Expose Apex Methods to Lightning Web Components

To expose an Apex method to a Lightning web component, the method must be static and either global or public. Annotate the method with @AuraEnabled. These requirements are the same as using an Apex method in an Aura component. The @AuraEnabled annotation means that you can call a method from an Aura component or a Lightning web component. Even though @AuraEnabled seems unexpected for a Lightning web component, using the annotation means that you can use the same Apex code in both programming models.

Use the Wire Service to Call an Apex Method

If an Apex method is cacheable (it doesn't mutate data), you can invoke it from a component via the wire service. You must annotate the method with @AuraEnabled(cacheable=true).



Don't @wire an Apex method that creates, updates, or deletes data.





```
5      return [SELECT Id, Name FROM Account
6              WHERE AnnualRevenue > 1000000];
```

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Work with Salesforce Data

This Javascript code calls the Apex `getAccounts()` method using the wire service.

```
1  import { LightningElement, wire } from 'lwc';
2  import getAccounts from
3  '@salesforce/apex/MyAccountController.getAccounts';
4  export default class HelloApexAccounts extends LightningElement
5  {
6      accounts=[];
7      @wire(getAccounts, {})
8      wiredAccounts({ error, data }) {
9          if (error) {
10             this.error = error;
11          } else if (data) {
12             this.accounts = data;
13          }
14      }
15  }
```



Note the syntax of the statement that imports `getAccounts`.

```
1  import getAccounts from
    '@salesforce/apex/MyAccountController.getAccounts';
```



- `MyAccountController` is the name of the Apex class
- `getAccounts` is the @AuraEnabled method that we're importing

Call an Apex Method Directly

If an Apex method mutates (creates, updates, or deletes) data and therefore isn't cacheable, you must call the method directly in code.





```

5     contacts;
6     getContacts() {
7         getContactList()
8         then(result => {
9             this.contacts = result;
10        })
11        .catch(error => {
12            // Handle error. Details in error.message.
13        });
14    }
15 }

```

Lightning Web Components for Aura Developers (/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Work with Salesforce Data

The import statement looks the same as the wire service example. However, the syntax to call the method doesn't have an @wire annotation. Instead, getContactList() returns a Promise object that resolves on success with the list of contacts.

Work with External APIs

Working with external APIs in a Lightning web component is similar to Aura components. By default, you can't make calls to third-party APIs from JavaScript code in Lightning web components. Add a remote site as a CSP Trusted Site to allow JavaScript component code to load assets from and make API requests to that site's domain.

Aura components and Lightning web components have the same restrictions around running JavaScript code from third-party sites. To use a third-party library, you must upload the library as a static resource.

Resources

- [Lightning Aura Components Developer Guide: Calling a Server-Side Action](https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/controllers_server_actions_call.htm) (https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/controllers_server_actions_call.htm)
- [Lightning Aura Components Developer Guide: Manage Trusted URLs](https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/security_trusted_urls_manage.htm) (https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/security_trusted_urls_manage.htm)
- [Lightning Aura Components Developer Guide: Using External JavaScript Libraries](https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/external_javascript_libraries.htm) (https://developer.salesforce.com/docs/atlas.en-us.lightning.meta/lightning/external_javascript_libraries.htm)





[library/documentation/lwc/lwc.data_get_user_input](#))

- [Lightning Web Components Developer Guide: lightning/ui*Api Wire Adapters and Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello/&trailmix_slug=salesforce-platform-developer\)](#)
[library/documentation/lwc/lwc.reference_ui_api](#))

[Work with Salesforce Data](#)
• [Lightning Web Components Developer Guide: createRecord\(recordInput\)](#)
(https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.reference_create_record)

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

1 Which base component creates a form to view, edit, and create Salesforce records?

A lightning-record-view-form

B lightning-record-form

C lightning-swiss-army-knife

D lightning-form

2 What should you use @wire for in a JavaScript file?





To update data

[Lightning Web Components for Aura Developers \(/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-for-aura-developers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



To delete data

Work with Salesforce Data

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points

Learn

- Badges
- Trails
- Trailmixes
- Superbadges
- Trailhead Academy
- Career Paths

Certifications

- Certifications
- Maintain Certifications
- Verify Certifications

Community

- Trailblazer Community
- Events
- Agentblazers
- Salesforce Developers
- Salesforce Admins
- Code of Conduct
- Report Illegal Content (EU)

Extras

- Trailhead Login
- Sales Enablement
- Trailblazer Store
- Salesforce Help

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)



☒ Your Privacy Choices

